Random Numbers

Randomness is highly useful in applications, whether you're working with simulations or cryptography. PHP offers a range of functions to generate random data for multiple use cases.

Let's look at two of the earliest to be implemented in PHP (version 4!): `rand()` and `mt_rand()`:

```
var_dump(rand()); // random int between 0 and a system-dependent maximum
var_dump(mt_rand(0, 6)); // random int, min 0 and max 6
var_dump(rand(-1, 1)); // random int min -1 and max 1
var_dump(getrandmax()); // varies with platform
var_dump(mt_getrandmax()); // varies with platform
```

```
Output:
int(549992859)
int(4)
int(-1)
int(2147483647)
int(2147483647)
```

The final two lines will output `int(2147483647)` on a non-Windows platform. `mt_rand()` uses a different, faster algorithm than `rand()`, and since PHP 7.1 `mt_rand()` has replaced `rand()`. (`rand()` is still a valid function name but is now an alias of `mt_rand()`).

If a random float between 0 and 1 is required, the function `lcg_value()` will provide it for you:

```
var_dump(lcg_value());
```

```
Output:
float(0.2606030773622016)
```

Note that when specifying min and max value arguments, they must lie between 0 and the value of `getrandmax()` or risk impacting the randomness of the result. Herein lies a problem with random numbers generated by machines. Generating a random number on a machine designed for calculating and ordering number sequences is not a simple task. In the case of `rand()`, `mt_rand()`, and `lcg_value()`, the values of the output are potentially predictable and therefore not suited for any kind of cryptographic use.